

Aplicação prática dos conceitos

Projeto: **Desenvolvimento de um Aplicativo Móvel de E-commerce**

Objetivo: Criar um aplicativo de e-commerce para facilitar a compra de produtos para os usuários.

1. Planejamento Inicial (Sprint 0)

- **Objetivo do Sprint 0:** Planejar e definir a visão do projeto, criar o backlog inicial, selecionar a equipe e a ferramenta de gestão do projeto (ex.: Jira ou Trello).

· Tarefas:

- Definir os requisitos iniciais do aplicativo.
- Montar a equipe (desenvolvedores, designers, testers, PO - Product Owner).
- Estabelecer o tempo das Sprints (2 semanas, por exemplo).

2. Sprints

Cada Sprint é composta de ciclos curtos de desenvolvimento, onde o time trabalha em funcionalidades específicas. A cada final de Sprint, o time realiza uma **revisão** e uma **retrospectiva**.

Sprint 1 - Funcionalidade de Cadastro e Login

· **Objetivo:** Desenvolver o fluxo de cadastro e login do usuário.

· Backlog da Sprint:

- Criação da tela de cadastro.
- Implementação do login com e-mail e senha.
- Validação de dados no frontend.
- Testes unitários para o login.

· Reuniões:

- **Daily Standups** diários para sincronização da equipe.
- **Revisão da Sprint:** Apresentação da funcionalidade pronta ao final da Sprint.
- **Retrospectiva da Sprint:** Análise do que funcionou bem e o que pode ser melhorado.

Sprint 2 - Funcionalidade de Carrinho de Compras

· **Objetivo:** Implementar a funcionalidade do carrinho de compras.

· Backlog da Sprint:

- Adicionar produtos ao carrinho.

- Excluir produtos do carrinho.
- Visualizar o total do carrinho.
- Implementação da lógica de estoque.

· **Reuniões:**

- Daily Standups para revisar o progresso.
- Revisão e Retrospectiva ao final da Sprint.

Sprint 3 - Integração de Pagamento

· **Objetivo:** Implementar a integração com sistema de pagamento (ex.: Stripe ou PayPal).

· **Backlog da Sprint:**

- Criação da tela de pagamento.
- Integração com a API de pagamento.
- Implementação de notificações de sucesso ou falha na transação.

· **Reuniões:**

- Daily Standups diários.
- Revisão da Sprint ao final.
- Retrospectiva para ajustes no processo.

3. Entrega Final

Ao final de várias Sprints, o aplicativo estará pronto para ser lançado. A equipe entregará a versão funcional do produto, testada e validada pelo cliente.

4. Exemplo de Fluxo do Scrum

- **Product Owner (PO):** Responsável por definir o backlog do produto e priorizar as funcionalidades.
- **Scrum Master:** Facilita o processo, removendo impedimentos e garantindo que a metodologia ágil seja seguida.
- **Equipe de Desenvolvimento:** Trabalha nas tarefas, desenvolvendo as funcionalidades conforme as prioridades definidas.

5. Benefícios do Ágil no Projeto

- **Adaptação Rápida:** Mudanças de requisitos podem ser incorporadas facilmente entre as Sprints.
- **Entrega Contínua:** Funcionalidades são entregues incrementadas a cada Sprint, garantindo valor contínuo para o cliente.
- **Feedback Contínuo:** O cliente pode testar o produto em cada fase e dar feedback imediato.

A **implementação de testes** em um projeto ágil, como o exemplo do

desenvolvimento de um aplicativo móvel, é fundamental para garantir a qualidade do produto e a confiança nas funcionalidades. Os testes devem ser realizados de forma contínua durante o desenvolvimento, com foco na **automação de testes** para facilitar a validação rápida a cada iteração (Sprint).

Tipos de Testes a Serem Implementados

1. Testes Unitários

- **Objetivo:** Testar unidades específicas do código (como funções ou métodos) isoladamente.
- **Exemplo:** Testar uma função de cálculo de preço no carrinho de compras.
- **Ferramentas:** JUnit (Java), XCTest (iOS), Mocha/Chai (JavaScript), etc.

2. Testes de Integração

- **Objetivo:** Validar a interação entre diferentes componentes do sistema (por exemplo, a integração entre o frontend e o backend).
- **Exemplo:** Verificar se a funcionalidade de pagamento está integrando corretamente com a API do sistema de pagamento (ex.: Stripe, PayPal).
- **Ferramentas:** Postman, REST Assured, etc.

3. Testes de Interface do Usuário (UI)

- **Objetivo:** Garantir que a interface do usuário está funcionando corretamente e proporcionando a experiência esperada.
- **Exemplo:** Verificar se a tela de login é exibida corretamente e se os botões estão funcionando.
- **Ferramentas:** Selenium, Appium, Espresso (para Android), XCTest (para iOS).

4. Testes de Aceitação (Testes de Funcionalidade)

- **Objetivo:** Validar que as funcionalidades do produto atendem às necessidades do cliente e requisitos do backlog.
- **Exemplo:** Testar a funcionalidade de adicionar um item ao carrinho de compras.
- **Ferramentas:** Cucumber (BDD), FitNesse, etc.

5. Testes de Performance

- **Objetivo:** Testar o comportamento do sistema sob condições de carga (por exemplo, como o aplicativo responde com um grande número de usuários ou transações simultâneas).
- **Exemplo:** Testar a velocidade de carregamento da tela de pagamento.
- **Ferramentas:** JMeter, LoadRunner, Gatling.

6. Testes de Regressão

- **Objetivo:** Garantir que alterações no código não afetem funcionalidades existentes.
- **Exemplo:** Após a implementação de uma nova funcionalidade de pagamento, é necessário verificar se o processo de login continua funcionando corretamente.
- **Ferramentas:** Ferramentas de automação de testes como Selenium, Appium, etc.

Fluxo de Implementação de Testes no Ciclo Ágil

1. Planejamento da Sprint:

- O **Product Owner (PO)**, em conjunto com a equipe de desenvolvimento, define as funcionalidades que precisam ser entregues.
- As tarefas de teste (unitários, de integração, etc.) são planejadas e adicionadas ao **backlog da sprint**.

2. Desenvolvimento com Testes:

- A equipe de desenvolvimento cria as funcionalidades e implementa os testes unitários enquanto desenvolve o código.
- A equipe utiliza a **integração contínua (CI)** para rodar os testes automaticamente sempre que há uma nova mudança no código.

3. Execução dos Testes:

- Os testes são executados regularmente após cada mudança no código, seja em um servidor de integração contínua ou localmente.

4. Revisão e Retrospectiva:

- Durante a **revisão da Sprint**, são apresentadas as funcionalidades desenvolvidas e os testes realizados.
- Na **retrospectiva**, a equipe discute o desempenho dos testes, como melhorar a cobertura de testes ou como tornar a execução mais eficiente.

Exemplo de Fluxo de Trabalho de Testes Ágeis:

1. Desenvolvimento: O desenvolvedor cria uma funcionalidade (ex.: cadastro de usuário).

- Implemente testes unitários para validar a lógica de cadastro.

2. Commit: O código é enviado para o repositório.

- A ferramenta de **integração contínua** (como Jenkins ou GitHub Actions) detecta a mudança e executa os testes.

3. Resultados dos Testes: Os resultados dos testes são reportados. Caso algum teste falhe, o desenvolvedor recebe um alerta para corrigir o erro.

4. Revisão da Sprint: Os testes são revisados como parte da **demonstração da funcionalidade** durante a Sprint Review.

5. Feedback e Ajustes: A equipe recebe feedback do cliente sobre os testes e ajustes são feitos conforme necessário.

Benefícios da Implementação de Testes em um Ambiente Ágil

- **Deteção precoce de erros:** Testes contínuos ajudam a identificar problemas rapidamente, antes que se tornem críticos.
- **Redução de retrabalho:** Com testes automatizados, é possível verificar rapidamente se novas mudanças não impactaram o que já estava funcionando.
- **Aumento da qualidade do produto:** Com a automação e testes regulares, o produto se mantém confiável e de alta qualidade durante o desenvolvimento contínuo.
- **Feedback rápido:** Testes fornecem feedback imediato sobre o funcionamento das funcionalidades, ajudando a manter o ritmo do desenvolvimento ágil.

O **Kanban** é uma metodologia ágil muito eficaz para o gerenciamento visual de projetos. Ele foca na entrega contínua e na melhoria do fluxo de trabalho, permitindo que as equipes se concentrem nas tarefas de forma eficiente e organizada, sem sobrecarga. Aplicar o Kanban no gerenciamento de um projeto envolve o uso de um **quadro Kanban** e a implementação de algumas práticas chave. Aqui está como você pode aplicar o Kanban no gerenciamento de um projeto:

1. Criando o Quadro Kanban

O quadro Kanban é a ferramenta central para visualização do fluxo de trabalho. Ele pode ser físico (com post-its em uma parede) ou digital (ferramentas como Trello, Jira, ClickUp, etc.). O quadro geralmente é dividido em colunas que representam os diferentes estados de uma tarefa, como:

- **Backlog (A Fazer):** Tarefas a serem realizadas.
- **Em Progresso:** Tarefas que estão sendo trabalhadas no momento.
- **Em Revisão/Testes:** Tarefas concluídas, mas precisam ser verificadas ou testadas.
- **Concluído:** Tarefas finalizadas e entregues.

Essas colunas podem ser ajustadas dependendo do fluxo específico de cada equipe ou projeto. Por exemplo, se você estiver desenvolvendo um aplicativo,

pode incluir colunas como “Desenvolvimento” e “Aprovação do Cliente”.

2. Limitação de Trabalho em Progresso (WIP - Work In Progress)

Uma das práticas essenciais do Kanban é a **limitação do trabalho em progresso (WIP)**. Isso significa que você deve definir um número máximo de tarefas que podem estar em “Em Progresso” ao mesmo tempo. Isso ajuda a evitar a sobrecarga da equipe e mantém o foco nas tarefas prioritárias.

Exemplo de Limite de WIP:

- “Em Progresso” pode ter no máximo 3 tarefas de cada vez. Isso força a equipe a terminar tarefas antes de começar novas.

3. Dividindo o Trabalho em Tarefas Menores

No Kanban, é importante dividir grandes tarefas em unidades menores e mais gerenciáveis. Cada tarefa representada no quadro deve ser o mais detalhada possível para que a equipe saiba exatamente o que precisa ser feito.

Exemplo:

- Em vez de colocar “Desenvolver o aplicativo”, divida a tarefa em ações específicas como:
 - “Criar tela de login”
 - “Implementar validação de senha”
 - “Integrar API de pagamento”

4. Visualização e Acompanhamento do Fluxo de Trabalho

O quadro Kanban permite que você visualize o **fluxo de trabalho** e identifique onde as tarefas estão paradas. Isso facilita a identificação de gargalos e a tomada de decisões rápidas para resolver problemas.

Exemplo:

- Se muitas tarefas estão empacadas na coluna “Em Revisão”, pode indicar que há um problema no processo de revisão (como falta de revisores ou uma demora nas validações).

5. Gerenciamento de Prioridades

No Kanban, as prioridades podem ser ajustadas a qualquer momento. Você pode adicionar cartões para novas tarefas no **Backlog** e movê-las para as colunas conforme elas se tornam mais urgentes.

Exemplo:}

- Se um cliente relatar um bug crítico, você pode mover a tarefa de “Concluído” para “Em Progresso” imediatamente, dando prioridade a essa

tarefa.

6. Reuniões de Acompanhamento (Kanban Meetings)

Embora o Kanban não exija reuniões fixas como no Scrum, realizar **reuniões regulares** para discutir o progresso do projeto pode ser muito útil. Estas reuniões devem ser curtas e focadas, por exemplo, uma reunião semanal de **Revisão de Fluxo** para ver como o projeto está avançando.

Exemplo de Reunião:

- **Revisão de Fluxo:** Revisão semanal para analisar a quantidade de trabalho em progresso, os gargalos no fluxo e as prioridades para a próxima semana.

7. Melhorias Contínuas (Kaizen)

Uma das filosofias por trás do Kanban é a melhoria contínua. O time deve buscar sempre maneiras de **melhorar o fluxo de trabalho** e otimizar o processo. Isso pode ser feito por meio de reuniões de retrospectiva ou por observação do fluxo e das métricas de desempenho.

Exemplo de Melhoria:

- Se um certo tipo de tarefa sempre fica parado na coluna “Em Revisão”, o time pode explorar formas de acelerar essa fase (revisores adicionais, novos critérios, etc.).

8. Métricas Kanban

O Kanban utiliza métricas para analisar e melhorar o desempenho da equipe. Algumas métricas importantes são:

- **Lead Time:** Tempo total para concluir uma tarefa (do início até a conclusão).
- **Cycle Time:** Tempo gasto para concluir uma tarefa desde que ela entrou na coluna “Em Progresso”.
- **Throughput:** Quantidade de tarefas concluídas dentro de um determinado período.

Essas métricas ajudam a identificar áreas que precisam de melhorias e a monitorar a eficiência do fluxo de trabalho.

Exemplo Prático de Kanban em um Projeto

Vamos considerar um projeto de desenvolvimento de um aplicativo de e-commerce. O fluxo de trabalho no quadro Kanban poderia ser o seguinte:

1. Backlog (A Fazer):

- Criar tela de login
- Integrar pagamento com PayPal

- Criar tela de produtos
- Testar a funcionalidade de carrinho de compras

2. Em Progresso:

- Criar tela de login (limite de WIP = 2, então outras tarefas não podem ser movidas para essa coluna até que a tarefa atual seja finalizada).

3. Em Revisão/Testes:

- Tela de login (aguardando aprovação do QA)
- Funcionalidade de carrinho (aguardando testes de usabilidade)

4. Concluído:

- Tela de login implementada e testada
- Funcionalidade de pagamento integrada

Com esse fluxo, o time pode acompanhar facilmente o andamento de cada tarefa e garantir que o trabalho seja feito de maneira eficiente e sem sobrecarga.

Benefícios do Kanban no Gerenciamento de Projetos

- **Visualização clara:** Todos os membros da equipe sabem o que está sendo feito, quem está fazendo o quê e o que ainda precisa ser feito.
- **Flexibilidade:** O Kanban é adaptável e permite ajustar as prioridades rapidamente conforme o projeto evolui.
- **Redução de desperdício:** Com a limitação de WIP e foco no fluxo contínuo, o Kanban ajuda a minimizar tarefas inacabadas e redundantes.
- **Melhoria contínua:** O sistema de Kanban incentiva a equipe a melhorar constantemente o processo e a eficiência.

A **análise e implementação de medidas de segurança** em um projeto é crucial para proteger os dados e a infraestrutura de um sistema, principalmente em projetos ágeis que estão sempre em evolução. Essas medidas devem ser abordadas de forma contínua, já que a segurança não deve ser tratada apenas como um passo final, mas como uma parte integrante de todo o ciclo de vida do desenvolvimento.

1. Análise de Riscos e Planejamento de Segurança

Antes de implementar qualquer medida de segurança, é fundamental realizar uma **análise de riscos** para identificar vulnerabilidades e potenciais ameaças no projeto. Isso ajuda a determinar quais medidas de segurança são mais adequadas.

Passos:

- **Identificação dos ativos:** O que precisa ser protegido? (dados sensíveis, infraestrutura, usuários).
- **Avaliação das ameaças:** Que ameaças podem impactar o projeto? (ataques de hackers, falhas de software, roubo de dados).
- **Avaliação da probabilidade e impacto:** Com que frequência essas ameaças podem ocorrer e qual o impacto de cada uma?
- **Estabelecimento de objetivos de segurança:** O que você deseja alcançar com a segurança (confidencialidade, integridade, disponibilidade)?

2. Implementação de Controle de Acesso

Garantir que apenas usuários autorizados possam acessar dados ou funcionalidades sensíveis é uma das primeiras medidas a serem implementadas.

Medidas:

- **Autenticação forte (ex.: 2FA):** Exigir múltiplos fatores de autenticação (senha + código enviado por SMS ou aplicativo).
- **Controle de privilégios:** Atribuir permissões mínimas necessárias para cada usuário (Princípio do Menor Privilégio).
- **Gerenciamento de sessões:** Controlar o tempo de expiração de sessões e permitir que os usuários se desconectem de maneira segura.

Exemplo: Implementação de autenticação de dois fatores (2FA) em um aplicativo para garantir que apenas usuários legítimos tenham acesso ao sistema.

3. Proteção contra Injeção de Código e Vulnerabilidades Comuns

A proteção contra vulnerabilidades conhecidas como **injeção de código** (SQL Injection, Cross-Site Scripting - XSS, etc.) é essencial para prevenir ataques que podem comprometer o sistema.

Medidas:

- **Validação de entrada:** Validar e sanitizar todas as entradas de dados (inputs de usuários, parâmetros de URL, etc.).
- **Uso de ORM (Object-Relational Mapping):** Em vez de escrever SQL diretamente, utilize ORMs para prevenir SQL injection.
- **Escapamento de dados (para XSS):** Garantir que dados recebidos de usuários sejam tratados corretamente antes de serem exibidos no navegador, evitando a execução de scripts maliciosos.

4. Criptografia de Dados

A criptografia é essencial para proteger dados em trânsito (quando estão sendo enviados) e em repouso (quando estão armazenados).

Medidas:

- **Criptografia SSL/TLS:** Para proteger a comunicação entre o cliente e o servidor, garantindo que dados sensíveis (como senhas e informações de pagamento) sejam criptografados durante a transmissão.
- **Criptografia de dados em repouso:** Criptografar dados armazenados em bancos de dados e arquivos, garantindo que, mesmo se acessados indevidamente, os dados não possam ser lidos.

5. Monitoramento e Detecção de Anomalias

Monitorar constantemente o sistema em busca de comportamentos anormais pode ajudar a identificar possíveis tentativas de invasão ou falhas de segurança rapidamente.

Medidas:

- **Logs de segurança:** Registre todas as atividades relacionadas à segurança (tentativas de login, acessos administrativos, mudanças de configuração).
- **Análise de tráfego de rede:** Monitore tráfego de rede em busca de padrões incomuns que possam indicar um ataque, como **DDoS (ataques de negação de serviço)** ou tráfego malicioso.
- **Sistema de alertas:** Configure alertas para situações críticas (tentativas de login falhadas, acesso não autorizado, etc.).

Ferramentas:

- **SIEM (Security Information and Event Management):** Ferramentas como Splunk, ELK Stack ou Datadog podem ajudar a monitorar e correlacionar eventos de segurança.

6. Atualizações e Patches de Segurança

Manter o sistema atualizado é uma das medidas mais simples, mas frequentemente negligenciadas, para prevenir ataques.

Medidas:

- **Patches de segurança:** Aplique atualizações e patches de segurança assim que forem disponibilizados pelos fornecedores de software.
- **Monitoramento de vulnerabilidades:** Utilize ferramentas para escanear o sistema em busca de vulnerabilidades conhecidas (ex.: **OWASP ZAP, Nessus**).

7. Testes de Segurança e Avaliações Contínuas

Realizar testes de segurança regularmente ajuda a identificar novas

vulnerabilidades e a garantir que as medidas de segurança implementadas sejam eficazes.

Testes e Avaliações:

- **Testes de penetração (Penetration Testing):** Simular ataques cibernéticos para identificar pontos fracos no sistema.
- **Auditorias de segurança:** Contratar especialistas externos para realizar auditorias e avaliar o nível de segurança do sistema.
- **Análise de código estático:** Utilizar ferramentas de análise de código para detectar vulnerabilidades antes da execução.

8. Treinamento e Conscientização da Equipe

Treinar a equipe sobre boas práticas de segurança ajuda a reduzir os riscos de erros humanos, como senhas fracas ou falhas na implementação de medidas de segurança.

Medidas:

- **Treinamento regular em segurança:** Realizar treinamentos sobre segurança da informação, políticas de segurança e melhores práticas.
- **Conscientização sobre phishing:** Ensinar os membros da equipe a reconhecer e-mails fraudulentos e tentativas de phishing.

GitHub da Disciplina

Confira os códigos e scripts usados ao longo da disciplina no repositório dela no GitHub: <https://github.com/FaculdadeDescomplica/Pratica-Integradora-com-Metodos-Ageis>

Conteúdo Bônus

Um curta educativo sobre ágil que simula a transformação de uma equipe tradicional em uma equipe ágil é o “Agile: The Movie” de 2013. E para assistir entrevistas com especialistas da indústria sobre o impacto das metodologias ágeis no dia a dia em um contexto ágil, o documentário “The Agile Revolution”. Outro documentário que pode ser assistido para aprender sobre a adoção do kanban em equipes reais e os benefícios no fluxo de trabalho é “Kanban: The Movie” de 2014; para aprender sobre análise e implementação de medidas de segurança, um documentário sobre um vírus chamado Stuxnet, criado para sabotar instalações nucleares, é “Zero Days”.

Referências Bibliográficas

HIGHSMITH, James A. Agile project management: creating innovative products. Boston: Addison-Wesley, 2010.

ANDERSON, David James. Kanban: successful evolutionary change for your technology business. Sequim, WA: Blue Hole Press, 2010.

OWASP FOUNDATION. OWASP top ten web application security risks. Baltimore, MD: OWASP Foundation, 2021. Disponível em: <https://owasp.org/www-project-top-ten/>. Acesso em: 25 abr. 2025.

SCHWABER, Kenneth S.; SUTHERLAND, Jeffrey Victor. Scrum Guide: o guia definitivo do Scrum – as regras do jogo. [Scrum.org](https://scrumguides.org), 2020. Disponível em: <https://scrumguides.org>. Acesso em: 25 abr. 2025.

Atividade Prática 12 – Aplicação prática dos conceitos

Título da Prática: Aplicação prática dos conceitos

Objetivos: Aplicar conceitos de metodologias ágeis

Materiais, Métodos e Ferramentas: Para realizar esta prática vamos ler atentamente o conteúdo da unidade

Atividade Prática

1. Uma empresa de tecnologia fechou contrato para desenvolver um novo aplicativo e quer garantir entregas rápidas, com possibilidade de ajustes conforme o cliente testa as funcionalidades. A equipe de desenvolvimento decidiu adotar metodologias ágeis para conduzir o projeto, garantindo maior colaboração e flexibilidade.

Explique os **benefícios de utilizar metodologias ágeis** na condução de projetos e cite pelo menos **duas práticas comuns** que podem ser aplicadas nesse contexto, justificando como elas ajudam a alcançar os objetivos do projeto.

2. Durante o desenvolvimento de um sistema web, a equipe percebeu que a ausência de uma estratégia clara de testes resultou em vários bugs críticos na versão entregue ao cliente. O gerente de qualidade sugeriu a implementação de uma abordagem estruturada de testes para garantir maior confiabilidade do sistema.

Explique a importância da implementação de testes durante o desenvolvimento de software e cite pelo menos duas práticas ou tipos de

testes que devem ser implementados, justificando sua relevância para garantir a qualidade do sistema.

3. Uma empresa de e-commerce sofreu recentemente um ataque que comprometeu dados sensíveis de seus clientes. Após o incidente, a equipe de desenvolvimento e segurança iniciou um processo de análise e implementação de medidas de segurança para evitar futuras vulnerabilidades e proteger as informações.

Explique a importância da análise e implementação de medidas de segurança em sistemas de software e cite pelo menos duas medidas que podem ser aplicadas, justificando como elas contribuem para proteger o sistema.

Gabarito Atividade Prática

1. Benefícios de utilizar metodologias ágeis:

- Entregas frequentes e incrementais de valor ao cliente.
- Maior flexibilidade para mudanças de requisitos.
- Comunicação e colaboração contínuas entre equipe e cliente.
- Foco na satisfação do cliente através da entrega de software funcional.

Duas práticas comuns:

- Reuniões diárias (Daily Scrum):

Breves encontros para alinhamento das atividades e identificação de impedimentos.

Garante comunicação eficaz e rápida resolução de problemas, acelerando o progresso do projeto.

- Revisão e Retrospectiva da Sprint:

Avaliação ao final de cada ciclo sobre o que foi entregue e como melhorar o processo.

Favorece a melhoria contínua e o ajuste das entregas de acordo com o feedback do cliente.

2. Importância da implementação de testes:

Os testes asseguram que o software funcione conforme o esperado, prevenindo falhas, reduzindo custos com correções e aumentando a satisfação do cliente. A implementação de uma estratégia de testes desde o

início do desenvolvimento garante maior robustez e segurança do produto.

Duas práticas ou tipos de testes:

Testes de unidade:

- Validam partes isoladas do código, como funções e métodos.

Facilitam a detecção precoce de erros e ajudam a manter a qualidade conforme o sistema evolui.

- Testes de integração:

Verificam se os diferentes módulos do sistema interagem corretamente.

Garantem que as funcionalidades funcionem bem em conjunto, evitando falhas de comunicação entre componentes.

Importância da análise e implementação de medidas de segurança:

A segurança da informação é fundamental para proteger dados sensíveis, garantir a confiabilidade do sistema, preservar a reputação da empresa e evitar prejuízos financeiros e legais. A análise de segurança permite identificar vulnerabilidades e definir medidas preventivas.

Duas medidas de segurança:

- Criptografia de dados:

Transforma informações sensíveis em formatos ilegíveis para pessoas não autorizadas.

Garante a confidencialidade dos dados, especialmente em sistemas que armazenam informações pessoais ou financeiras.

- Validação e sanitização de entradas:

Processos que verificam e tratam as entradas de dados fornecidas pelos usuários.

Previnem ataques como SQL Injection e Cross-Site Scripting (XSS), que exploram falhas de validação para comprometer o sistema.